



Rust

Cristiano Damiani Vasconcellos
cristiano.vasconcellos@udesc.br

Tuplas

Tuplas são coleções de diferentes tipos de um tamanho fixo, por exemplo, duplas, triplas, quadruplas, etc. Os elementos das tuplas podem ser acessados pelos índices de suas posições: 0, 1, 2, ...

```
fn main() {  
    let mut a = (2, 'A');  
    let b = (true, 2.5, 1);  
    a.0 = 10;  
    println!("{a:?}");  
    println!("{b:?}");  
}
```

Smart Pointer Box<T>

Box é um dos tipos de ponteiros inteligentes de Rust que permite a alocação de espaços no *heap*.

```
fn main() {  
    let mut a: Box<i32> = Box::new(5);  
    let mut b = Box::new(10);  
    *a = 20;  
    troca (a, b);  
    println!("{a}-{b}"); // Erro  
}  
  
fn troca(mut a:Box<i32>, mut b:Box<i32>) {  
    let aux = *a;  
    *a = *b;  
    *b = aux;  
    println!("troca:{a} {b}");  
}
```

Smart Pointer Box<T>

```
fn main() {  
    let mut a: Box<i32> = Box::new(5);  
    let mut b = Box::new(10);  
    *a = 20;  
    troca (&mut a, &mut b);  
    println!("{a}-{b}");  
}  
  
fn troca(a:&mut Box<i32>, b:&mut Box<i32>) {  
    let aux = **a;  
    **a = **b;  
    **b = aux;  
}
```

Listas

```
#[derive (Debug)]
enum Lista {
    Cons (i32, Lista),
    Nil
}

fn main() {
    let list = Cons(1, Cons(2, Cons(3, Nil))); //Erro
}
```

```
#[derive (Debug)]
enum Lista{
    Cons (i32, Box<Lista>),
    Nil
}

use Lista::{*};

fn main() {
    let l1 = Cons(1, Box::new(Cons(2, Box::new(Cons(3,
        Box::new(Nil))))));

    println!("{l1:?}");
}
```

Listas

```
#[derive (Debug)]
enum NoLista{
    Cons {cab:i32, cauda:Box<NoLista>},
    Nil
}
#[derive (Debug)]
struct Lista {
    cabeca:Box<NoLista>
}
use NoLista::{*};
impl Lista {
    fn new() -> Self {
        Lista {cabeca : Box::new(Nil)}
    }
    fn ins_cab(&mut self, e:i32) {
        self.cabeca = Box::new(Cons {cab:e, cauda:self.cabeca});
        /*^^^^^^^^ Erro: Não é possível
                    mover um conteúdo
                    emprestado */
    }
}
```

Métodos

Métodos são um caso especial de funções associadas que recebem como primeiro parâmetro o dado para o qual foi chamado, por meio de self. Existem basicamente 3 formas que pode ser declarado:

- self – Valor, quando um dado é passado por valor, esse valor é movido para a nova área e a anterior perde a propriedade do dado (exceto em caso de cópia).
- &self - Referência compartilhada (imutável), compartilha a referência para área onde o dado está armazenado, pode acessar o dado mas não possui a sua propriedade.
- &mut self – Referência mutável, recebe uma referência para área onde o dado está armazenado e empresta a propriedade do dado de forma exclusiva. Nenhuma outra parte do código pode acessar essa área e seu valor pode ser modificado. A única coisa que você não pode fazer com uma referência mutável é movê-la sem substituí-la.

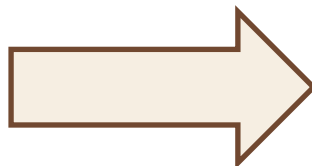

```
#[derive (Debug)]
enum NoLista{
    Cons {cab:i32, cauda:Box<NoLista>},
    Nil
}
#[derive (Debug)]
struct Lista {
    cabeca:Box<NoLista>
}
use std::mem::replace;
use NoLista::{*};
impl Lista {
    fn new() -> Self {
        Lista {cabeca : Box::new(Nil)}
    }
    fn ins_cab(&mut self, e:i32) {
        let n = Box::new(Cons {cab:e, cauda: replace(&mut self.cabeca,
                                                    Box::new(Nil))});
        self.cabeca = n;
    }
}
```

```
fn remover_cab (&mut self) -> Option<i32>{
    match *replace (&mut self.cabeca, Box::new(Nil)) {
        Nil => None,
        Cons {cab:x, cauda:xs}=> {
            self.cabeca = xs;
            Some(x)
        }
    }
}
```

Tipos Genéricos

Permitem definir funções e estruturas polimórficas com tipos parametrizados que são resolvidos em tempo de compilação:

```
#[derive (Debug)]
enum NoLista{
    Cons {cab:i32, cauda:Box<NoLista>},
    Nil
}
#[derive (Debug)]
struct Lista {
    cabeca:Box<NoLista>
}
...
let mut l1 = Lista::new();
```



```
#[derive (Debug)]
enum NoLista<T>{
    Cons {cab:T, cauda:Box<NoLista<T>>},
    Nil
}
#[derive (Debug)]
struct Lista {
    cabeca:Box<NoLista<T>>
}
...
let mut l1:Lista<i32> = Lista::new();
l1.ins_cab(20);
```

Listas com Tipos Genéricos

```
#[derive (Debug)]
enum NoLista<T>{
    Cons {cab:T, cauda:Box<NoLista<T>>},
    Nil
}

#[derive (Debug)]
struct Lista<T> {
    cabeca:Box<NoLista<T>>
}

use std::mem::replace;
use NoLista::{*};
impl<T> Lista<T> {
    fn new() -> Self {
        Lista {cabeca : Box::new(Nil)}
    }
    fn ins_cab(&mut self, e:T) {
        let n = Box::new(Cons {cab:e, cauda: replace(&mut self.cabeca, Box::new(Nil))});
        self.cabeca = n;
    }
}
```

Listas com Tipos Genéricos

```
impl<T:Ord> Lista<T> {  
    fn ins_ordenado (&mut self, e:T) {  
        let p = replace(&mut self.cabeca, Box::new(Nil));  
        self.cabeca = Self::ins_ord(p, e);  
    }  
  
    fn ins_ord (p:Box<NoLista<T>>, e:T) -> Box<NoLista<T>>{  
        match *p {  
            Nil => Box::new(Cons {cab:e, cauda:p}),  
            Cons {cab:x, cauda:xs} => if e < x {Box::new(Cons {cab:e,  
                                                            cauda:Box::new(Cons {cab:x,cauda:xs}})})  
                else {Box::new (Cons {cab:x,  
                                    cauda:Self::ins_ord (xs, e)}})}  
        }  
    }  
}
```

Listas com Tipos Genéricos

```
#[derive (Debug, PartialEq, Eq, PartialOrd, Ord)]
enum Semana {Dom, Seg, Ter, Qua, Qui, Sex, Sab}

fn main() {
    let mut l = Lista::new();

    l.ins_ordenado(Qua);
    l.ins_ordenado(Seg);
    l.ins_ordenado(Dom);
    println!("{l:?}");
}
```

Listas com Tipos Genéricos

```
#[derive (Debug)]
struct No<T>
{
    elem:T,
    prox:Option<Box<T>>
}
```

```
#[derive (Debug)]
struct Lista<T> {
    cabeca:Box<No<T>>
}
```

Dúvida ??

```
fn remover_cauda_loop(&mut self) {  
    let mut p = &mut self.cabeca;  
    loop {  
        match **p {  
            Cons {cauda: ref mut xs, .. } => {  
                match **xs {  
                    Cons {..} => {  
                        p = xs;  
                    }  
                    Nil => {  
                        break;  
                    }  
                }  
            }  
            Nil => return,  
        }  
    }  
    *p = Box::new(Nil);  
}
```